## 4.- ARBOLES MULTICAMINO

Hasta ahora el análisis se ha limitado a los árboles en que cada nodo tienen como máximo dos descendientes o hijos, es decir, a los árboles binarios. Esto resulta perfectamente apropiado si, por ejemplo, se quieren representar relaciones familiares en las que cada persona se encuentre relacionada con sus padres. Pero si la relación es a la inversa necesitamos una estructura que asocie a cada padre un número arbitrario de hijos. A estas estructuras se les llama árboles multicamino o n-arios. Este último término viene del inglés n-ary, dónde n es el grado máximo de los nodos del árbol, por ejemplo, si n es igual a 2 al árbol se le llama binario, para n igual a 3 se le llama terciario o ternario, para n igual a 4 se le llama cuaternario, y así sucesivamente.

### 4.1.- TDA Arbol Multicamino. Definición

Son los árboles definidos al principio del capítulo, es decir, cada nodo tienen un número arbitrario de hijos y se sigue manteniendo el orden entre los nodos.

### 4.2.- Implementaciones de Arboles Multicamino

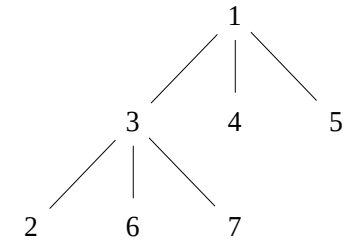
Las siguientes son formas tradicionales para representar un árbol multicamino en memoria:

- Mediante arreglos
- Mediante lista de hijos
- Basada en árboles binarios

#### 4.2.1.- Mediante arreglos

a) Sea A un árbol cuyos nodos tienen como nombres 1, 2, ..., n. Tal vez la representación más sencilla para manejar la operación Padre, sin utilizar enlaces entre nodos, sea un arreglo unidimensional, indexada por los nombres de los nodos, y cuyo contenido A[i] sea el nombre del padre del nodo i.

Sea el siguiente árbol A: El arreglo con que representamos este árbol sería:



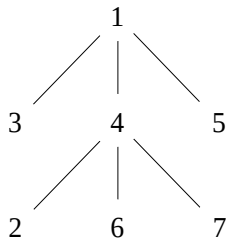

| 0 | 3 | 1 | 1 | 1 | 3 | 3 |
|---|---|---|---|---|---|---|
| [1] | [2] | [3] | [4] | [5] | [6] | [7] |

El padre del nodo raíz lo representamos con el valor 0 que es un valor nulo.
Esta representación se puede realizar porque en un árbol cada nodo tiene un padre único, por lo tanto esta implementación permite hallar el padre de un nodo en un tiempo constante. Esto da lugar a que se pueda obtener un camino ascendente en el árbol, es decir de un nodo a su padre, de éste a su padre y así sucesivamente, en un tiempo proporcional al número de nodos del camino.

Esta representación no facilita las operaciones que requieren información de los hijos. Dado un nodo n, resulta difícil determinar sus hijos o su altura. Además, esta representación no especifica el orden de los hijos de un nodo. Se puede imponer un orden artificial, por ejemplo, numerando los hijos de cada nodo después de numerar al padre, y numerando los hijos en orden ascendente, de izquierda a derecha.

b) Otra realización sería aquella en la que cada nodo se compone de la raíz y un arreglo de árboles con tantas componentes como el máximo de hijos de los nodos del árbol.

Supongamos el siguiente árbol: Su representación sería:




| elemento | Hijos | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 3 | 4 | 5 | | | | |
| 4 | 2 | 6 | 7 | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |

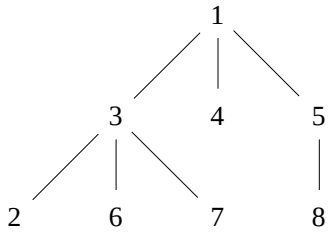
Esta implementación requiere una estimación del número de hijos y si fuese muy variable desperdiciaría mucha memoria.

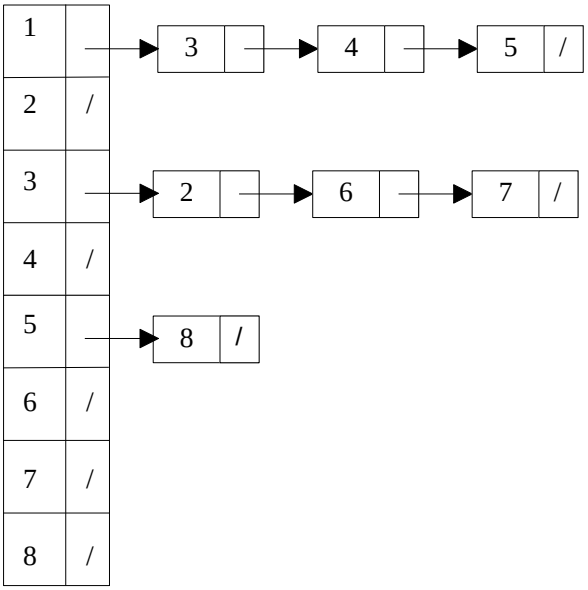
**4.2.2.- Mediante listas de hijos**

a) Otra manera importante y útil de representar árboles consiste en formar una lista de los hijos de cada nodo. Las listas se pueden implementar con cualquiera de los métodos vistos, pero como el número de hijos que cada nodo puede tener es variable, es más apropiada la implementación de éstas mediante referencias.

En definitiva, lo que se va a tener, es un arreglo indexado por los nombres de los nodos, y donde el contenido de cada casilla va a ser la etiqueta del nodo y una lista enlazada de árboles hijos de izquierda a derecha.

Supongamos el árbol A: Su representación mediante listas de hijos es la siguiente:



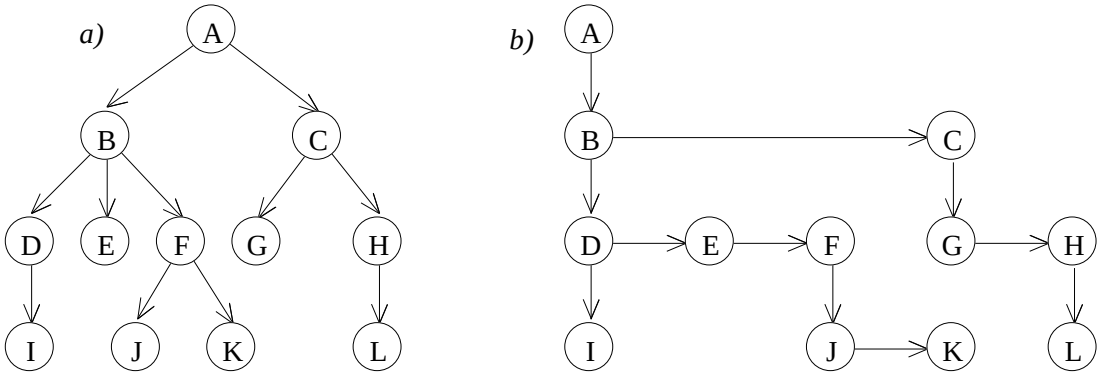
Al tener el arreglo tantas casillas como nodos tiene el árbol, se produce un desperdicio de espacio ya que los nodos hojas no tienen hijos y por lo tanto sus casillas correspondientes, dentro de este arreglo, no se usan. Pero este problema no se puede solucionar ya que en principio no se sabe qué nodos son interiores y cuáles hojas. Pero se puede observar que con esta implementación es fácil conocer el número de hijos de cada nodo.
Para conocer el padre de un nodo no resulta tan fácil ya que es necesario recorrer todas las listas hasta encontrar dicho nodo, en cuyo caso su padre será el nodo correspondiente a esa casilla de encabezamiento, suponiendo que hay un sólo árbol; en otro caso la búsqueda tendría que hacerse recursivamente. En resumen, el recorrido en profundidad (hacia arriba) se complica.

b) Otra alternativa, es utilizar una lista de listas. Hacerlo totalmente dinámico. En vez de representar los nodos de origen en un vector, se crea un nodo de una lista por cada uno, siempre que tenga hijos. De esta manera, no se desperdicia memoria en aquellos casos donde un nodo no posee hijos.

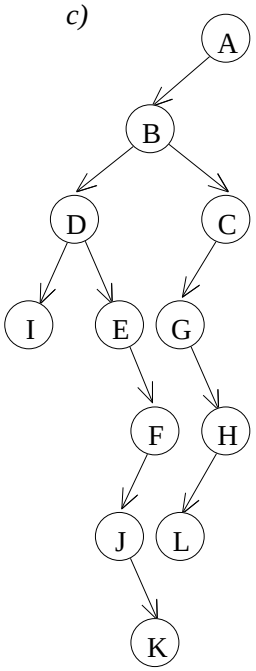
**4.2.3.- Basada en Arboles Binarios**
Los árboles generales quedan representados por árboles binarios de manera que el hijo izquierdo es el hijo más a la izquierda y el hijo derecho es el siguiente hermano hacia la derecha.
La principal ventaja de esta representación es su simplicidad. De hecho, esta es la representación más usada para árboles generales.
Aquí se establecerán los mecanismos necesarios para convertir un árbol general en un árbol binario. Los pasos que se deben aplicar para lograr la conversión del árbol general al árbol binario son los siguientes:

1. Deben enlazarse los hijos de cada nodo en forma horizontal (los hermanos).
2. Debe enlazarse en forma vertical el nodo padre con el hijo que se encuentra más a la izquierda. Además, debe eliminarse el vínculo de ese padre con el resto de sus hijos.
3. Debe rotarse el diagrama resultante, aproximadamente 45 grados hacia la izquierda, y así se obtendrá el árbol binario correspondiente.

Ejemplo:



*c)*




Para todo nodo de un árbol binario, obtenido a partir de un árbol general, debe cumplirse lo siguiente:

1. En la rama derecha de cada nodo, excepto el nodo raíz, si ésta es distinta del vacío se encuentra un nodo que era hermano de éste en el árbol general.
2. En la rama izquierda de cada nodo, si ésta es distinta del vacío, se encuentra un nodo que era hijo de éste en el árbol general.

**4.3.- Arboles Multicamino B**

Un área muy práctica de aplicación de árboles multicamino es la construcción y mantenimiento de árboles de búsqueda a gran escala, donde es preciso realizar inserciones y supresiones de elementos y en situaciones en que la memoria principal es demasiado costosa o no suficientemente grande como para ser utilizada como almacenamiento permanente.

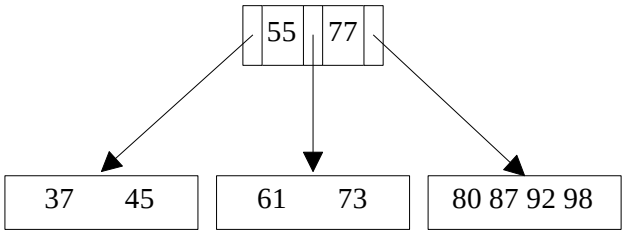
Supóngase que el volumen de información de un árbol es tal que no cabe en memoria principal y debe ser guardado en un dispositivo de almacenamiento secundario, como por ejemplo, un disco. Si se utilizase un árbol binario como estructura para almacenar los elementos, la búsqueda de un elemento requerirá muchos pasos (en el peor de los casos, tantos como altura tenga el árbol), cada uno de los cuales necesita un acceso al disco. Una forma de reducir el número de accesos sería almacenar los elementos en bloques, llamados páginas, de manera que el acceso a una página representa ahora un acceso al disco, ahorrando así un tiempo considerable. Este es el objetivo de los árboles multicamino. Una página es un nodo del tipo $p_0(k_1p_1)(k_2p_2)...(k_np_n)$ donde $k_i$ son claves únicas tales que $k_i < k_{i+1}$ y $p_i$ son referencias a subárboles. Todas las claves del subárbol $p_i$ son mayores que $k_i$ y menores que $k_{i+1}$.

Debemos tener presente que un acceso a disco es extremadamente lento si lo comparamos con un acceso a memoria; es por lo tanto de primordial interés la reducción del número de accesos a disco. Si se accede a un elemento que está almacenado en memoria secundaria, también se puede acceder a un grupo completo de elementos sin que sea preciso mucho esfuerzo adicional. Esto es, cuesta lo mismo leer 4 bytes que 512 bytes, pues la unidad mínima de transferencia (bloque) suele ser de 512 bytes. Esto sugiere agrupar la información del árbol en bloques o páginas.

Un ejemplo de árbol multicamino en donde además se controla el crecimiento desequilibrado es el árbol-B. Por lo tanto, su principal característica es su buen rendimiento en cuanto a operaciones de búsqueda, aunque veremos que esto supone un coste en las operaciones de inserción y borrado.

Los árboles-B fueron propuestos por Bayer y McCreight, en el año 1970. Propusieron que:

1. Cada nodo tiene como máximo n hijos.
2. Cada nodo (excepto raíz y hojas) tiene como mínimo n/2 hijos.
3. La raíz tiene al menos 2 hijos, sino es un nodo hoja.
4. Todos los nodos hoja aparecen al mismo nivel y el camino de la raíz a las hojas tiene la misma longitud.
5. Un nodo no hoja con k hijos contiene k-1 elementos almacenados (claves).
6. Los hijos que cuelgan de la raíz ($r_1$, ···, $r_m$) tienen que cumplir ciertas condiciones:
    1. El primero tiene valor menor que $r_1$.
    2. El segundo tiene valor mayor que $r_1$ y menor que $r_2$, etc.
    3. El último hijo tiene valor mayor que $r_m$.
7. Dentro de cada hoja las claves se encuentran ordenadas en forma creciente.




**4.3.1.- Búsqueda en Arboles Multicamino B**

Esta organización supone una extensión natural de los árboles binarios de búsqueda. El proceso de búsqueda en árboles-B es una generalización del proceso de búsqueda en árboles binarios de búsqueda.

1. Debe tenerse en memoria principal la página sobre la cual vamos a buscar.

Considérese la página de la figura, en la cual cada $k_i$ representa una clave y cada $p_i$ un apuntador, y el elemento a buscar x. Si m es suficientemente grande, se puede utilizar la búsqueda binaria. Si m es bastante pequeña, una búsqueda secuencial será suficiente.

2. Si la búsqueda es infructuosa se estará en una de las siguientes situaciones:
    2.1. $k < x < k_{i+1}$ para $1 <= i < m$. La búsqueda continúa en la página $p_i$.
    2.2. $k_m < x$. La búsqueda continúa en la página $p_m$.
    2.3. $x < k_1$. La búsqueda continúa en la página $p_0$.

Si en algún caso la referencias es null, es decir, si no hay página descendiente, entonces no hay ningún elemento x en todo el árbol y se acaba la búsqueda.

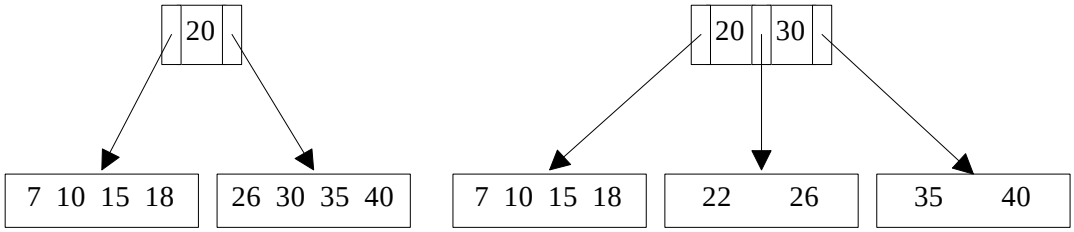
**4.3.2.- Inserción en Arboles Multicamino B**

Los árboles-B tienen un comportamiento particular, diferente al resto de los árboles estudiados anteriormente. Todas las hojas están al mismo nivel y por lo tanto cualquier camino desde la raíz hasta alguna de las hojas tiene la misma longitud. Por otra parte, los árboles-B crecen de abajo hacia arriba, desde las hojas hasta la raíz.

Para insertar un elemento en un árbol-B, primero debe localizarse la página donde corresponde insertar el elemento. Una vez encontrada, pueden darse dos casos: que la página tenga espacio para más elementos, o que esté llena. Una página está llena, cuando el número de claves es mayor que la aridad restada en 1, es decir: Nº de claves > n-1 (n es el orden u aridad del árbol)

Si la página no está llena, el elemento se inserta en el lugar que le corresponde. En cambio, si el número de elementos o claves de la página es mayor que la aridad - 1 (página llena), el proceso tendrá consecuencias directas en la estructura del árbol. La página afectada se divide en 2, distribuyéndose los m+1 elementos equitativamente entre las dos páginas. El elemento del medio sube a la página antecesora. Si la página antecesora se desborda nuevamente, entonces se tendrá que repetir el proceso correspondiente al caso anterior. El proceso de propagación puede llegar incluso hasta la raíz, en dicho caso la altura del árbol puede incrementarse en una unidad.

Inserción de la clave 22 en un árbol-B




**4.3.3.- Borrado en Arboles Multicamino B**

Consiste en quitar un elemento del árbol sin que se violen los principios que definen a un árbol-B. Se distinguen dos casos:

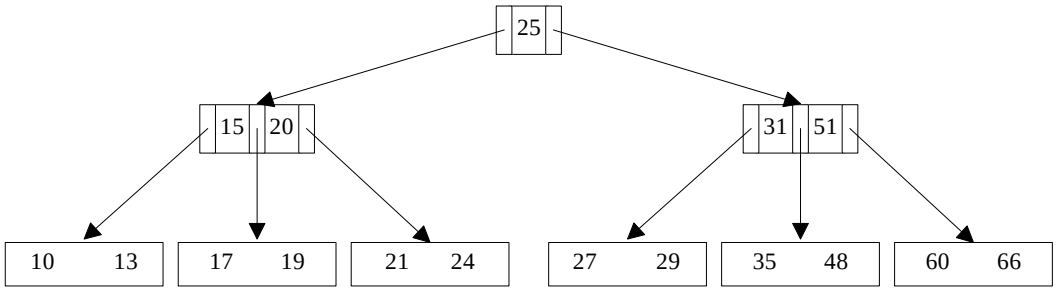
1. El elemento a borrar se encuentra en una página hoja, entonces simplemente se suprime.
2. La clave a borrar no se encuentra en una página hoja, entonces debe sustituirse por la clave que se encuentra más a la izquierda en el subárbol derecho o por la clave que se encuentra más a la derecha en el subárbol izquierdo.

En cualquiera de los dos casos verificamos:

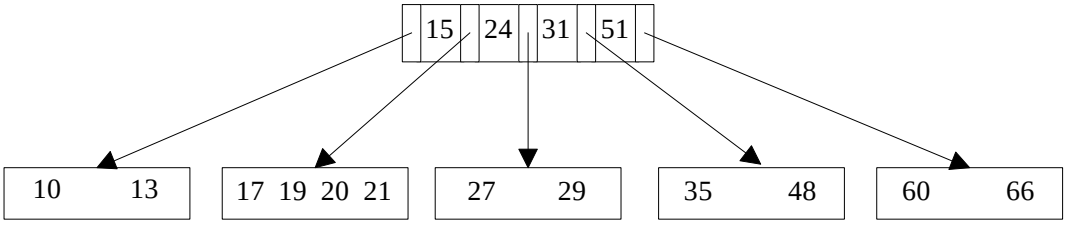
- Si el número de claves es mayor que el mínimo, termina la operación de borrado.
- Si el número de claves es igual al mínimo, debe bajarse la clave lexicográficamente adyacente de la página antecesora y fusionar las páginas que son descendientes directas de dicha clave.

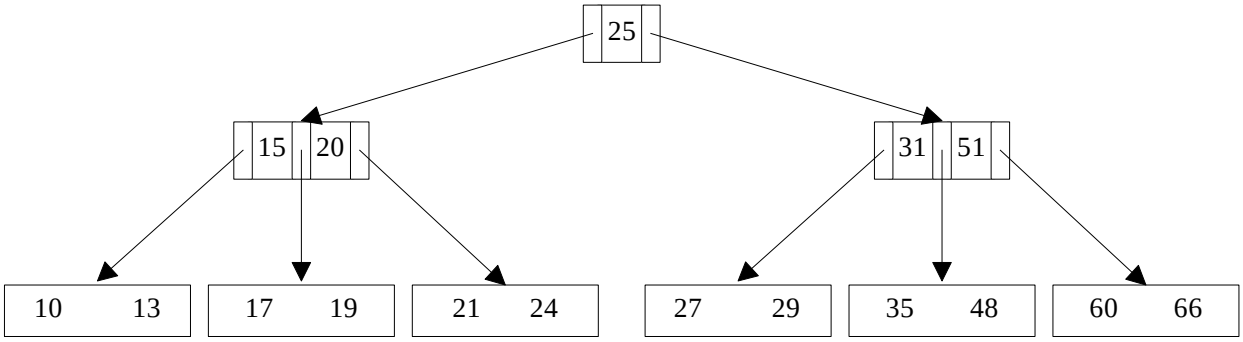
Cabe aclarar que el proceso de fusión de páginas puede propagarse incluso hasta la raíz, en cuyo caso la altura del árbol disminuye en una unidad.

Ejemplos:

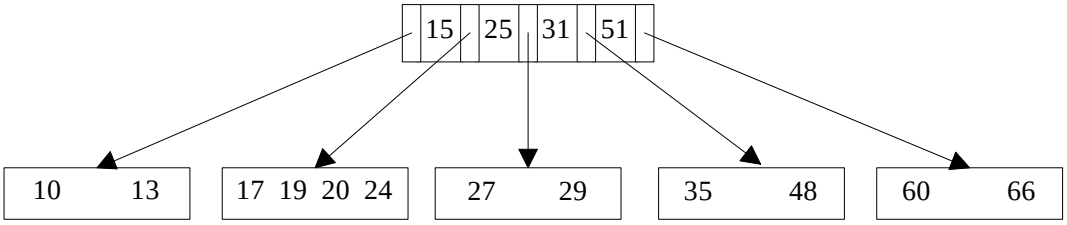



Eliminación de la clave 25






Eliminación de la clave 21




### 4.3.4.- Implementación de Arboles Multicamino B

```
import java.util.*;
class BTree
        {
        BNode root;
        int arity;

        BTree(int arity)
                {
                this.arity=arity;
                root=null;
                }

        void addKey(Object key)
                {
                if(root==null)
                        {
                        BNode newNode=new BNode(arity + 1);
                        Keys keys=new Keys(arity);
                        int aux=keys.size;
                        keys.addKey(key);
                        newNode.setData(keys);
                        root=newNode;
                        return;
                        }
                //find a leaf to insert key. The leaf will be referenced by currentNode
                BNode currentNode=root;
                Keys currentKeys=root.getData();
                int index;
                Stack nodes=new Stack();
                while(currentNode.degree()>0)
                        {
                        index=currentKeys.search(key);
                        if(index==-1)
                                return;
                        nodes.push(currentNode);
                        currentNode=currentNode.getChild(index);
                        currentKeys=currentNode.getData();
                        }
                index=currentKeys.search(key);
```

```
		if(index==-1)
			return;
		//insert key into currentNode
		int aux=currentKeys.size;
		currentKeys.addKey(key);
		//if a node contains arity number of keys, split the node to two nodes.
		//Repeat the step for its parent and other ancestors if necessary
		while (currentKeys.size==arity)
			{
			int mid=(arity + 1) / 2 - 1;
			key=currentKeys.keyAt(mid);
			currentKeys.deleteKey(key);
			BNode newNode=new BNode(arity + 1);
			Keys newKeys=new Keys(arity);
			newNode.setData(newKeys);
			//move keys from currentNode to newNode
			for(int i=0; i<(arity + 1) / 2 - 1; i++)
				{
				Object tempKey=currentKeys.keyAt(0);
				newKeys.addKey(tempKey);
				currentKeys.deleteKey(tempKey);
				}
			//move children
			if(currentNode.degree()>0)
				for(int i=0; i<=(arity + 1) / 2 - 1; i++)
					{
					BNode child=currentNode.getChild(0);
					newNode.addChild(child,i);
					currentNode.deleteChild(0);
					}
			if(currentNode==root)
				{
				root=new BNode(arity + 1);
				Keys tempKeys=new Keys(arity);
				tempKeys.addKey(key);
				root.setData(tempKeys);
				root.addChild(newNode,0);
				root.addChild(currentNode,1);
				return;
				}
			//add key and newNode into parent
			currentNode=(BNode)nodes.peek();
			nodes.pop();
			currentKeys=currentNode.getData();
			index=currentKeys.addKey(key);
			currentNode.addChild(newNode,index);
			}
		}

	boolean searchKey(Object key)
		{
		if(root==null)
			return false;
		BNode currentNode=root;
		Keys currentKeys=root.getData();
		int index=currentKeys.search(key);
		while(currentNode.degree()>0)
			{
			if(index==-1)
				return true;
			currentNode=currentNode.getChild(index);
			currentKeys=currentNode.getData();
			index=currentKeys.search(key);
			}
		if(index!=-1)
			return false;
		else
			return true;
		}
```

```
boolean deleteKey(Object key)
	{
	if(root==null)
		return false;
	BNode currentNode=root;
	Keys currentKeys=root.getData();
	int index=currentKeys.search(key);
	Stack nodes=new Stack();
	//searches for key in the BTree
	while(currentNode.degree()>0)
		{
		if(index==-1)
			return true;
		nodes.push(currentNode);
		currentNode=currentNode.getChild(index);
		currentKeys=currentNode.getData();
		index=currentKeys.search(key);
		}
	if(index==-1 && currentNode.degree()>0)
		{//key is in a non-leaf node
		currentKeys.deleteKey(key);
		index=currentKeys.search(key);
		nodes.push(currentNode);
		Keys tempKeys;
		if(index<currentNode.degree() - 1)
			{//finds inorder successor
			currentNode=currentNode.getChild(index + 1);
			while(currentNode.degree()>0)
				{
				nodes.push(currentNode);
				currentNode.getChild(0);
				}
			tempKeys=currentNode.getData();
			key=tempKeys.keyAt(0);
			}
		else
			{//finds inorder predecessor
			currentNode=currentNode.getChild(index);
			while(currentNode.degree()>0)
				{
				nodes.push(currentNode);
				currentNode=currentNode.getChild(currentNode.degree() - 1);
				}
			tempKeys=currentNode.getData();
			key=tempKeys.keyAt(tempKeys.size - 1);
			}
		currentKeys.addKey(key);
		tempKeys.deleteKey(key);
		currentKeys=tempKeys;
		}
	else
		{
		if(index==-1)
			currentKeys.deleteKey(key);
		else
			return false; //key is not in BTree
		}
	while(currentNode!=root && currentKeys.size<(arity + 1) / 2 - 1)
		{
		BNode parent=(BNode)nodes.peek();
		nodes.pop();
		Keys parentKeys=parent.getData();
		//find the index of currentNode in the children array of parentNode
		for(int i=0; i<parent.degree(); i++)
			{
			if(currentNode==parent.getChild(i))
				{
				index=i;
				break;
				}
			}
```

```
if(index>0)
        {// currentNode has left sibling
        BNode leftSibling=parent.getChild(index - 1);
        Keys leftKeys=leftSibling.getData();
        Object tempKey=parentKeys.keyAt(index - 1);
        parentKeys.deleteKey(tempKey);
        if(leftKeys.size>=(arity + 1) / 2)
                {//right rotation
                Object movedKey=leftKeys.keyAt(leftKeys.size - 1);
                leftKeys.deleteKey(movedKey);
                parentKeys.addKey(movedKey);
                currentKeys.addKey(tempKey);
                if(currentNode.degree()>0)
                        {
                        BNode movedNode=leftSibling.getChild(leftSibling.degree() - 1);
                        leftSibling.removeChild(movedNode);
                        currentNode.addChild(movedNode,0);
                        }
                return true;
                }
        else
                {//node merge
                leftKeys.addKey(tempKey);
                for(int i=0; i<currentKeys.size; i++)
                        {
                        leftKeys.addKey(currentKeys.keyAt(i));
                        if(currentNode.degree()>0)
                                {
                                BNode tempNode=currentNode.getChild(i);
                                leftSibling.addChild(tempNode,leftSibling.degree());
                                }
                        }
                if(currentNode.degree()>0)
                      leftSibling.addChild(currentNode.getChild(currentNode.degree() - 1));
                parent.deleteChild(index);
                currentNode=parent;
                currentKeys=parentKeys;
                continue;
                }
        }
else
        {//currentNode has right sibling
        BNode rightSibling=parent.getChild(index - 1);
        Keys rightKeys=rightSibling.getData();
        Object tempKey=parentKeys.keyAt(index - 1);
        parentKeys.deleteKey(tempKey);
        if(rightKeys.size>=(arity + 1) / 2)
                {//left rotation
                Object movedKey=rightKeys.keyAt(0);
                rightKeys.deleteKey(movedKey);
                parentKeys.addKey(movedKey);
                currentKeys.addKey(tempKey);
                if(currentNode.degree()>0)
                        {
                        BNode movedNode=rightSibling.getChild(0);
                        rightSibling.removeChild(movedNode);
                        currentNode.addChild(movedNode,currentNode.degree());
                        }
                return true;
                }
        else
                {
                currentKeys.addKey(tempKey);
                for(int i=0; i<currentKeys.size; i++)
                        {
                        currentKeys.addKey(rightKeys.keyAt(i));
                        if(currentNode.degree()>0)
                                {
                                BNode tempNode=rightSibling.getChild(i);
                                currentNode.addChild(tempNode);
                                }
```

```
                                                }
                                        if(currentNode.degree()>0)
                                            currentNode.addChild(rightSibling.getChild(rightSibling.degree() - 1));
                                        parent.deleteChild(index + 1);
                                        currentNode=parent;
                                        currentKeys=parentKeys;
                                        continue;
                                        }
                                }
                        }//while
                if(currentNode==root)
                        if(root.getData().size==0)
                                if(root.degree()>0)
                                        root=root.getChild(0);
                                else
                                        root=null;
                return true;
                }
        }

class BNode
        {
        Keys data;
        BNode[] children;
        int arity, size;
        BNode(int arity)
                {
                this.arity=arity;
                size=0;
                children=new BNode[arity];
                }

        BNode(int arity, Keys data)
                {
                this.arity=arity;
                size=0;
                children=new BNode[arity];
                this.data=data;
                }

        void setData(Keys data)
                {
                this.data=data;
                }

        Keys getData()
                {
                return data;
                }

        BNode getChild(int index)
                {
                if(index>=size || index<0)
                        return null;
                return (BNode)children[index];
                }

        BNode addChild(BNode data)
                {
                if(size==arity)
                        return null;
                else
                        {
                        Keys temp=data.getData();
                        BNode tempNode=new BNode(arity, temp);
                        children[size++]=tempNode;
                        return tempNode;
                        }
                }
```

```
BNode addChild(BNode data, int index)
        {
        Keys temp=data.getData();
        if(index<0 || index>=size)
                return addChild(data);
        for(int i=size; i>index; i--)
                children[i]=children[i - 1];
        BNode tempNode=new BNode(arity, temp);
        children[index]=tempNode;
        size++;
        return tempNode;
        }

BNode deleteChild(int index)
        {
        if(index<0 || index>=size)
                return null;
        BNode tempNode=(BNode)children[index];
        for(int i=index; i<size - 1; i++)
                children[i]=children[i + 1];
        size--;
        return tempNode;
        }

void removeChild(BNode data)
        {
        if(size>0)
                {
                int j;
                for(int i=0; i<size; i++)
                        if((BNode)children[i]==data)
                                deleteChild(i);
                }
        }

int degree()
        {
        return size;
        }
}
```

**class Keys**

```
        {
        int arity, size;
        Object[] keys;

        /*Keys()
                {
                this(3);
                }*/

        Keys(int arity)
                {
                this.arity=arity;
                size=0;
                keys=new Object[arity];
                }

        Object keyAt(int index)
                {
                if(index<0 || index>=size)
                        return null;
                return keys[index];
                }

        int search(Object key)
                {
                for(int i=0; i<size; i++)
                        {
                        int c=((String)key).compareTo((String)keys[i]);
                        if(c<0)
```

```
                                        return i;
                                if(c==0)
                                        return -1;
                                }
                        return size;
                        }

        int addKey(Object key)
                        {
                        int index=search(key);
                        if(index==-1)
                                return -1;
                        for(int k=size; k>index; k--)
                                keys[k]=keys[k - 1];
                        keys[index]=key;
                        size++;
                        return index;
                        }

        boolean deleteKey(Object key)
                        {
                        int index=-1;
                        for(int i=0; i<size; i++)
                                {
                                int c=((String)key).compareTo((String)keys[i]);
                                if(c<0)
                                        return false;
                                if(c==0)
                                        {
                                        index=i;
                                        break;
                                        }
                                }
                        if(index==-1)
                                return false;
                        for(int j=index + 1; j<size; j++)
                                keys[j - 1]=keys[j];
                        size--;
                        return true;
                        }
        }

import java.util.*;
class Transversal implements Enumeration
        {
        private Vector nodes;

        public Transversal(BTree AB)
                        {
                        nodes=new Vector();
                        if(AB.root!=null)
                                nodes.addElement(AB.root);
                        }

        public boolean hasMoreElements()
                        {
                        return (nodes.size()!=0);
                        }

        public Object nextElement()
                        {
                        BNode tempNode=(BNode)nodes.elementAt(0);
                        nodes.removeElementAt(0);
                        for(int i=0; i<tempNode.degree(); i++)
                                nodes.addElement(tempNode.getChild(i));
                        return tempNode;
                        }
        }
```

### 4.4.- Arboles Multicamino B+

Los árboles-B+ se han convertido en la técnica más utilizada para la organización de archivos indexados. La principal característica de estos árboles es que todas las claves se encuentran en las hojas (a diferencia de los árboles-B, en que las claves podían estar en las páginas intermedias) y por lo tanto cualquier camino desde la raíz hasta alguna de las claves tienen la misma longitud.

Formalmente se define un árbol-B+ de la siguiente manera:

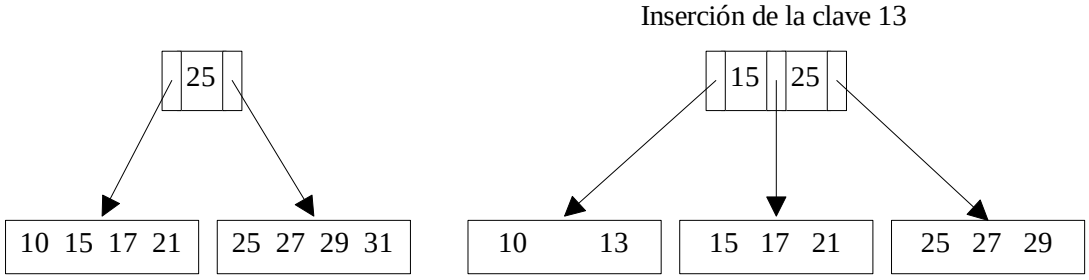
1. Cada página, excepto la raíz , contiene entre n y 2n elementos.
2. Cada página, excepto la raíz, tiene entre n+1 y 2n+1 descendientes. Se utiliza m para expresar el número de elementos por página.
3. La página raíz tiene al menos dos descendientes.
4. Las páginas hojas están todas al mismo nivel.
5. Todas las claves se encuentran en las páginas hojas.
6. Las claves de las páginas raíz e interiores se utilizan como índices.

#### 4.4.1.- Búsqueda en Arboles Multicamino B+

La operación de búsqueda es similar a la operación de búsqueda en árboles-B. Sin embargo puede suceder que al buscar un elemento éste se encuentre en una página raíz o interior, en cuyo caso debe continuarse la búsqueda por la rama derecha de dicha clave.

#### 4.4.2.- Inserción en Arboles Multicamino B+

La dificultad en el proceso de inserción se presenta cuando se desea insertar una clave en una página que se encuentra llena (m=2n). En este caso, la página afectada se divide en 2, distribuyéndose las m+1 claves de la siguiente forma: las n primeras claves en la página de la izquierda y las n+1 restantes claves en la página de la derecha. Una copia de la clave del medio sube a la página antecesora. Puede suceder que la página antecesora se desborde nuevamente, entonces tendrá que repetirse el proceso anterior.




Es importante notar que el desbordamiento en una página que no es hoja no produce duplicidad de claves. El proceso de propagación puede llegar hasta la raíz, en cuyo caso la altura del árbol puede incrementarse en una unidad.

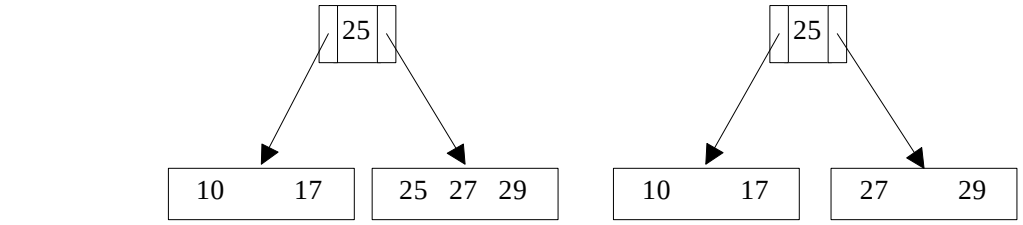
#### 4.4.3.- Borrado en Arboles Multicamino B+

En este caso las claves a eliminar siempre se encuentran en las páginas hojas. Se deben distinguir dos casos:

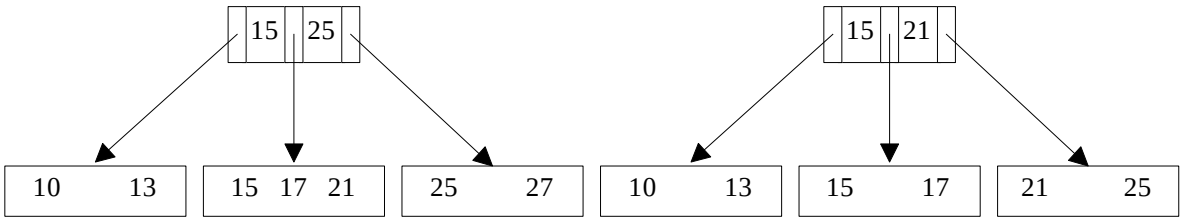
1. Si al eliminar una clave, m queda mayor o igual a n entonces termina la operación de borrado. Las claves de las páginas raíz o internas no se modifican por más que sean una copia de la clave eliminada en las hojas.
2. Si al eliminar una clave, m queda menor a n entonces debe realizarse una redistribución de claves, tanto en el índice como en las páginas hojas. Puede suceder que al eliminar una clave y al realizar una redistribución de las mismas, la altura del árbol disminuya en una unidad.

Ejemplos:

Eliminación de la clave 25




Eliminación de la clave 27




Eliminación de la clave 21

